

Software Design Specification

Bridging Healthcare Gaps with AI

Project Code:

BHG-AI-(25-26)

Project Advisors:

Dr. Hafiz Muhammad Faisal Shehzad
Mr. Aamir Zia

Project Manager:

Dr. Muhammad Ilyas

Project Team:

Aqsa Shehzad	BSCS51F22S028	(Team Lead)
Farwa Eiman Arif	BSCS51F22S014	(Team Member)
Sehrish Akram	BSCS51F22S019	(Team Member)
Ayesha	BSCS51F21R036	(Team Member)

Submission Date:

December 12, 2025

Project Manager's Signature

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	Bridging Healthcare Gaps with AI
Document	Software Design Specification
Document Version	1.0
Identifier	BHG-AI-(25-26)
Status	Draft
Author(s)	Aqsa Shehzad BSCS51F22S028 (Team Lead) Farwa Eiman Arif BSCS51F22S014 (Team Member) Sehrish Akram BSCS51F22S019 (Team Member) Ayesha BSCS51F21R036 (Team Member)
Approver(s)	PM
Issue Date	Sept. 15, 2025
Document Location	
Distribution	1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term	Description
ASP	Active Server Pages
DD	Design Specification
PM	Project Manager
AI	Artificial Intelligence
API	Application Programming Interface
BHG	Bridging Healthcare Gaps
CHI	Composite Health Index
CRUD	Create, Read, Update, Delete
E/R	Entity - Relationship
EMS	Emergency Medical Services
FCM	Firebase Cloud Messaging
HIPAA	Health Insurance Portability and Accountability Act
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
ML	Machine Learning
MVC	Model - View - Controller
NLP	Natural Language Processing
OCR	Optical Character Recognition
OOP	Object - Oriented Programming
PMC	Pakistan Medical Commission
REST	Representational State Transfer
SDK	Software Development Kit
SMS	Short Message Service
SOLID	Single Responsibility, Open - Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
UML	Unified Modeling Language
URI	Uniform Resource Identifier
WebRTC	Web Real - Time Communication
WSS	WebSocket Secure

Table of Contents

1. Introduction	4
1.1 Purpose of Document	4
1.2 Project Overview	5
1.3 Scope	5
2. Design Considerations	7
2.1 Assumptions and Dependencies	8
2.2 Risks and Volatile Areas	12
3. System Architecture	19
3.1 System Level Architecture	19
3.2 Sub-System / Component / Module Level Architecture	20
3.3 Sub-Component / Sub-Module Level Architecture (1...n)	21
4. Design Strategies	19
4.1 Strategy 1...n	19
5. Detailed System Design	30
6. References	43

1. Introduction

This section establishes the purpose, scope, and methodology of the design document for all stakeholders.

1.1 Purpose of Document

This document provides detailed architectural and design specifications for the HealthBridge.ai system. These specifications contains a blueprint for the development team that shows the system's architectural components, the components' interactions, the system's data models, and the design choices that are made.

1.1.1 Intended Audience

This document is intended for the following stakeholders:

- 1. Development Team:** Software engineers, frontend developers, backend developers, and AI/ML engineers who will implement the system based on this design.
- 2. Project Managers:** To track progress against design specifications and manage development timelines.
- 3. Quality Assurance Team:** To develop test cases and validate that implementation matches design specifications.
- 4. Database Administrators:** To understand data models and implement database schemas.
- 5. Advisors and Evaluators:** To assess the technical soundness of the design approach.

1.1.2 Design Methodology

This project employs an Object-Oriented Design (OOD) methodology, adhering to SOLID principles and design patterns.

The design approach includes:

- **Layered Architecture:** Separation of concerns through presentation, business logic, and data access layers
- **Component-Based Design:** Modular, reusable components with well-defined interfaces
- **Service-Oriented Architecture:** Independent services communicating via APIs
- **Domain-Driven Design:** Business logic organized around healthcare domain concepts
- **Microservices Principles:** Scalable, independently deployable service modules

The design follows industry best practices including:

- SOLID principles for maintainable code
- Design patterns (Factory, Repository, Strategy, Observer)
- RESTful API design principles
- Asynchronous processing for scalability
- Caching strategies for performance
- Security-first approach

1.2 Project Overview

System Description HealthBridge.ai is an AI-powered healthcare platform designed to bridge the healthcare accessibility gap in Pakistan. The system addresses critical healthcare challenges including limited access to medical professionals, long waiting times, language barriers, and lack of health literacy among the general population.

1.2.1 Core Functionality

The system provides the following key capabilities:

1. AI Health Assistant (Primary Feature)

- Intelligent conversational interface powered by Natural Language Processing
- Symptom analysis and preliminary health guidance
- Medical terminology explanation in simple language (English and Urdu)
- Four-level urgency triage system (Green/Yellow/Red/Emergency)
- Context-aware conversations with memory of previous interactions
- Personalized recommendations based on user's medical history

2. Medical Report Analysis

- OCR-based text extraction from uploaded reports (PDF, JPG, PNG)
- Automated analysis of test results against normal ranges
- Plain language explanations of medical findings
- Abnormal value detection with severity classification
- Alerts for critical values requiring immediate attention

3. Doctor Consultation System

- Searchable directory of verified doctors by specialty and location
- Real-time appointment booking with slot management
- Integrated video/audio consultation via WebRTC
- Payment integration with Pakistani payment methods (JazzCash, EasyPaisa)
- Automated reminders via SMS, email, and push notifications

4. Medication Management

- Medication tracking with dosage and frequency information
- Scheduled reminders for medication adherence
- Adherence tracking and reporting
- Drug interaction warnings

5. Multi-Language Support

- Full interface and AI responses in English and Urdu
- Real-time language switching
- Culturally appropriate health information

1.2.2 Design Approach

The system design follows a three-tier architecture with clear separation of concerns:

Presentation Layer (Frontend):

- Responsive web application built with React.js/Next.js
- Mobile-first design approach
- Real-time updates via WebSocket connections
- Progressive Web App (PWA) capabilities

Business Logic Layer (Backend):

- RESTful API services built with Node.js/Express
- Microservices architecture for scalability
- Service orchestration and API gateway pattern
- Business rule validation and enforcement

Data Layer:

- PostgreSQL for relational data (users, appointments, medical records)
- Redis for caching and session management
- Google Cloud Storage for medical report files
- Vector database for AI/ML embeddings

AI/ML Layer:

- Pre-trained medical NLP models fine-tuned on healthcare data
- Symptom analysis engine with medical knowledge base
- Triage classification system with urgency level detection
- OCR service for medical report text extraction

1.2.3 Key Design Principles

1. *Modularity*: Independent, loosely-coupled modules for maintainability
2. *Scalability*: Horizontal scaling capability for handling increased load
3. *Security*: End-to-end encryption, secure authentication, data privacy
4. *Reliability*: Fault tolerance, error handling, graceful degradation
5. *Performance*: Caching, lazy loading, optimized database queries
6. *Maintainability*: Clean code, comprehensive documentation, design patterns
7. *Usability*: Intuitive UI/UX, accessibility standards, dual-language support

1.3 Scope

The scope section defines what features are included, excluded, and the operational boundaries of the system.

1.3.1 In-Scope Deliverables:

1. AI Health Assistant (Core Feature: Chatbot for patient assistance)
2. Natural language health queries (English & Urdu)
3. Symptom assessment, preliminary guidance & medical terminology explanation
4. Context-aware conversation with memory
5. Four-level urgency triage (Green/Yellow/Red/Emergency)
6. Automatic specialist recommendation
7. Chat-based interface
8. Image/PDF upload with OCR text extraction and plain language explanations of results
9. Interface in English & local languages and In-app language switching
10. Specialty & location-based doctor matching
11. Live video calling & real-time chat
12. Payment gateway integration, calendar sync & appointment reminders
13. Appointment history tracking
14. Abnormal value alerts and AI-powered pattern detection
15. Medical imaging diagnosis, emergency dispatch.

1.3.2 Out-of-scope Exclusions:

16. Test ordering, integration with wearables, Bluetooth devices, or IoT sensors.
17. EHR, hospital system, insurance, pharmacy, or lab integration.
18. HIPAA/GDPR certification, FDA approval, or clinical trials.
19. Health forums, social sharing, gamification, family accounts, or genetic test analysis.
20. Medical Diagnosis with the Health Assistant

1.3.3 System Boundaries

The system operates within the following boundaries:

1.3.3.1 Geographic Scope:

- Primary target: Pakistan
- Language support: English and Urdu
- Payment methods: Pakistani payment gateways (JazzCash, EasyPaisa, Bank Transfers)
- Doctor verification: Pakistan Medical Commission (PMC) standards

1.3.3.2 User Scope:

- Patients seeking health information and consultations
- Licensed doctors providing telemedicine services
- System administrators for platform management

1.3.3.3 Technical Scope:

- Web-based application (responsive design for desktop, tablet, mobile)
- Cloud-hosted infrastructure
- Third-party API integrations limited to: payment gateways, video calling SDKs, SMS/email services, OCR services.

1.3.3.4 Data Scope:

- User account information
- Medical profiles and health history
- Conversation logs with AI assistant
- Medical reports uploaded by users
- Appointment records
- Consultation notes (doctor-created)

2. Design Considerations

This section outlines design-specific assumptions and dependencies that guide architectural decisions.

2.1 Assumptions and Dependencies

Design-level assumptions and dependencies establish the foundation for technology choices and integration strategies.

2.1.1 Design-Level Assumptions

Thirteen key assumptions span technology stack, infrastructure, data availability, and user behavior expectations.

Technology Stack Assumptions

It is assumed that the chosen technology stack will remain supported and updated by their respective vendors during the project timeline.

1. Frontend Technology

- **Assumption:** React.js/Next.js framework will be used for web development
- **Rationale:** Provides component-based architecture, server-side rendering, and excellent developer ecosystem
- **Design Impact:** Component-based UI design, state management architecture

2. Backend Technology

- **Assumption:** Node.js with Express for API services
- **Rationale:** Asynchronous processing capability, extensive library support for healthcare
- **Design Impact:** RESTful API design, microservices architecture feasibility

3. Database Technology

- **Assumption:** PostgreSQL for relational data, Redis for caching
- **Rationale:** ACID compliance, JSON support, excellent performance
- **Design Impact:** Relational data modeling, caching strategy design

4. AI/ML Framework

- **Assumption:** Python-based ML frameworks (TensorFlow, PyTorch, or Hugging Face Transformers)
- **Rationale:** Extensive pre-trained medical models available, strong community support
- **Design Impact:** Separate AI service architecture, API-based communication

Infrastructure Assumptions

It is assumed that required servers, hosting environments, and network resources will be available and stable throughout the project lifecycle.

5. Cloud Hosting

- **Assumption:** Cloud infrastructure (Google Cloud) will be available
- **Rationale:** Scalability, managed services, global availability
- **Design Impact:** Containerized deployment design, auto-scaling configuration

6. Network Connectivity

- **Assumption:** Users have minimum 3G mobile data or WiFi connectivity
- **Rationale:** Required for real-time video consultation and API communication
- **Design Impact:** Progressive loading, data compression

7. Video Infrastructure

- **Assumption:** WebRTC is supported by target browsers
- **Rationale:** Modern browsers support WebRTC for video calls
- **Design Impact:** Browser-based video consultation design, fallback mechanisms

Data and Content Assumptions

It is assumed that all required datasets and content will be available, with quality, accuracy, completeness and in usable formats.

8. Medical Knowledge Base

- **Assumption:** Curated medical knowledge base data will be available for AI training
- **Rationale:** Required for accurate symptom analysis and health guidance
- **Design Impact:** Knowledge base schema design, update mechanism design

9. OCR Accuracy

- **Assumption:** OCR services can achieve accuracy on standard medical reports

- **Rationale:** Modern OCR technology performance benchmarks
- **Design Impact:** Error handling design, manual correction interface, confidence thresholds

10. Medical Report Formats

- **Assumption:** Most medical reports follow standard layouts with identifiable sections
- **Rationale:** Hospitals generally use standardized report templates
- **Design Impact:** Template-based parsing design, machine learning-based extraction

User Behavior Assumptions

It is assumed that users meet the minimum requirement to use the system and will provide accurate and timely input wherever it is required by the system.

11. Digital Literacy

- **Assumption:** Users can navigate basic web interfaces with minimal training
- **Rationale:** Growing smartphone adoption in Pakistan
- **Design Impact:** Intuitive UI design, guided workflows, help tooltips

12. Language Preference

- **Assumption:** Users primarily use Urdu or English
- **Rationale:** These are the most widely spoken languages in Pakistan
- **Design Impact:** Bilingual interface design, language detection, translation services

13. Device Access

- **Assumption:** Users have access to smartphones or computers with cameras
- **Rationale:** Required for uploading medical reports and video consultations
- **Design Impact:** Mobile-responsive design, file upload optimization, camera API integration

2.1.2 Design-Level Dependencies

Sixteen dependencies identify critical external services, AI/ML models, infrastructure, legal requirements, and development tools.

External Service Dependencies

The availability and performance of external (third-party) integrations are assumed to remain stable, function reliably during development and operation and maintain backend compatibility.

1. Payment Gateway Services

- *Dependency:* JazzCash and EasyPaisa API availability and reliability
- *Impact on Design:* Payment service abstraction layer, webhook handling, retry mechanisms
- *Mitigation:* Design payment gateway adapter pattern for easy switching, implement fallback payment methods

2. Video Calling SDK

- *Dependency:* Agora or Twilio SDK for WebRTC functionality
- *Impact on Design:* Video service wrapper design, token generation service, room management
- *Mitigation:* Design agnostic video service interface

3. SMS/Email Services

- *Dependency:* Third-party services for notification delivery (Twilio, SendGrid, AWS SES)
- *Impact on Design:* Notification service architecture, queue management, retry logic
- *Mitigation:* Multi-provider support design, local queue for failed deliveries

4. OCR Service

- *Dependency:* Google Cloud Vision
- *Impact on Design:* OCR service adapter design, rate limiting, cost optimization
- *Mitigation:* Caching mechanism, batch processing design, fallback OCR libraries

5. Cloud Storage

- *Dependency:* Google Cloud Storage
- *Impact on Design:* File storage service abstraction, URL generation, access control
- *Mitigation:* Design storage-agnostic interface, support multiple storage backends

AI/ML Model Dependencies

The solution depends on the availability of training data and pre-trained models in appropriate formats and relies support provided by external ML frameworks and libraries.

6. Pre-trained Language Models

- *Dependency:* Availability of pre-trained NLP models (BERT)
- *Impact on Design:* Model serving architecture, API design for inference
- *Mitigation:* Design model versioning system, A/B testing infrastructure

7. Medical Knowledge Datasets

- *Dependency:* Access to medical terminology databases, symptom-disease mappings
- *Impact on Design:* Knowledge base schema, update mechanisms
- *Mitigation:* Design extensible knowledge base, manual curation interface

8. Training Data

- *Dependency:* Availability of healthcare conversation data for model fine-tuning
- *Impact on Design:* Data collection pipeline, annotation tools, privacy-preserving techniques
- *Mitigation:* Synthetic data generation, transfer learning approaches

Infrastructure Dependencies

The system depends on cloud infrastructure to provide required compute, storage, and networking resources.

9. Database Scalability

- *Dependency:* Database performance under concurrent user load
- *Impact on Design:* Database indexing strategy, query optimization, connection pooling
- *Mitigation:* Read replicas design, caching layer, database sharding strategy

10. Server Capacity

- *Dependency:* Server resources for AI model inference and video streaming
- *Impact on Design:* Load balancing architecture, auto-scaling configuration
- *Mitigation:* Queue-based processing, request throttling, resource monitoring

Legal and Regulatory Dependencies

The project compliance depends on adherence to applicable data protection, privacy, and industry regulations and it also depends on licensing terms of third-party tools remaining unchanged during the project.

11. Doctor Verification

- *Dependency:* PMC (Pakistan Medical Commission) doctor verification process
- *Impact on Design:* Doctor onboarding workflow, verification status tracking
- *Mitigation:* Manual verification workflow as fallback

12. Data Privacy Compliance

- *Dependency:* Compliance with Pakistan's data protection laws
- *Impact on Design:* User consent management, data anonymization, audit logging
- *Mitigation:* Privacy-by-design approach, data minimization principles

13. Medical Disclaimer Requirements

- *Dependency:* Legal requirements for health information disclaimers
- *Impact on Design:* Disclaimer display logic, user acknowledgment tracking
- *Mitigation:* Prominent disclaimer placement, mandatory acceptance flows

Development Tool Dependencies

The tool compatibility is assumed between frameworks, libraries, and the selected programming languages.

14. Version Control System

- *Dependency:* Git and GitHub for source code management

- *Impact on Design:* Branching strategy, CI/CD pipeline design
- *Mitigation:* Local Git hosting option, alternative platforms

15. CI/CD Pipeline

- *Dependency:* Automated build and deployment tools (GitHub Actions, Jenkins, CircleCI)
- *Impact on Design:* Deployment architecture, environment management
- *Mitigation:* Manual deployment procedures as fallback

16. Monitoring and Logging

- *Dependency:* Monitoring tools (Prometheus, Grafana) and logging services (ELK stack)
- *Impact on Design:* Instrumentation design, log format standardization
- *Mitigation:* File-based logging, simple monitoring scripts

2.2 Risks and Volatile Areas

This section identifies potential risks that could impact the system design and areas where requirements or technology choices may change, along with mitigation strategies.

2.2.1 Technical Risks

We have analyzed five technical risks: AI accuracy, service failures, scalability, video quality, and OCR reliability.

1. AI Model Accuracy and Reliability

The AI health assistant may provide inaccurate or misleading health information, potentially causing patient harm or loss of trust.

Impact: High - Core functionality affected, safety concerns, legal liability

Likelihood: Medium

Mitigation (Design) Strategies:

- Implement confidence thresholding (reject responses below 0.5 confidence)
- Design escalation mechanism to human doctors for low-confidence scenarios
- Include prominent medical disclaimers in UI design
- Design feedback loop for continuous model improvement
- Implement multi-model ensemble approach for critical decisions

Volatile Design Area: AI model architecture, confidence thresholds, escalation triggers

Design Impact:

- Modular AI service design for easy model swapping
- Comprehensive logging of all AI predictions with confidence scores
- A/B testing infrastructure for model comparison

- Fallback mechanisms when AI service is unavailable

2. Third-Party Service Failures

Critical dependencies on payment gateways, video SDKs, SMS services, and OCR services may experience downtime or rate limiting.

Impact: High - Service interruptions, user dissatisfaction, revenue loss

Likelihood: Medium

Mitigation (Design) Strategies:

- Implement adapter pattern for all external services
- Design retry mechanisms with exponential backoff
- Implement circuit breaker pattern for failing services
- Design graceful degradation (e.g., audio-only if video fails)

Volatile Design Area: External service integration points, API contracts

Design Impact:

- Service abstraction interfaces allowing provider switching
- Queue-based processing for non-critical operations (notifications, emails)
- Health check endpoints for all external services
- Rate limiting and request throttling

3. Scalability Under High Load

System may experience performance degradation or crashes during peak usage (viral marketing, health emergencies, seasonal flu outbreaks).

Impact: High - User experience degradation, system unavailability

Likelihood: Medium

Mitigation (Design) Strategies:

- Design horizontally scalable architecture (stateless services)
- Implement caching at multiple layers (CDN, Redis, database query cache)
- Design asynchronous processing for non-real-time operations
- Implement database read replicas and connection pooling
- Design auto-scaling policies based on metrics

Volatile Design Area: Load balancing strategy, caching policies, database partitioning

Design Impact:

- Microservices architecture for independent scaling

- Message queue design for background jobs
- Database indexing strategy for query optimization

4. Video Consultation Quality Issues

Poor network conditions, device incompatibilities, or WebRTC bugs may result in low-quality video/audio consultations.

Impact: Medium-High - User dissatisfaction, consultation effectiveness reduced

Likelihood: High (in Pakistan's network conditions)

Mitigation (Design) Strategies:

- Implement adaptive bitrate streaming
- Design fallback to audio-only consultations
- Create network quality indicator in UI
- Design pre-consultation device/network checks
- Implement reconnection logic for dropped connections

Volatile Design Area: Video codec selection, bitrate thresholds, fallback triggers

Design Impact:

- Multiple video quality profiles (high, medium, low)
- Audio-only consultation mode
- Network diagnostic tools
- Session recording with recovery from disconnections

5. OCR Accuracy on Medical Reports

OCR may fail to accurately extract text from handwritten notes, poor-quality scans, or non-standard report formats.

Impact: Medium - Feature reliability affected, user trust issues

Likelihood: High

Mitigation (Design) Strategies:

- Design manual correction interface for OCR results
- Implement confidence scoring and flagging of low-confidence extractions
- Create guided upload process with image quality checks
- Implement OCR preprocessing (image enhancement, rotation correction)

Volatile Design Area: OCR service providers, confidence thresholds, preprocessing techniques

Design Impact:

- OCR result validation interface
- Image preprocessing pipeline design
- User feedback mechanism for incorrect OCR results

2.2.2 Security and Privacy Risks

Two critical security risks address data breaches and payment fraud with comprehensive protection strategies.

6. Data Breach and Unauthorized Access

Sensitive medical data, personal information, and payment details could be exposed through security vulnerabilities.

Impact: Critical - Legal liability, user trust loss, financial penalties, reputation damage

Likelihood: Medium

Mitigation (Design) Strategies:

- Implement end-to-end encryption for sensitive data
- Design role-based access control (RBAC) with principle of least privilege
- Create comprehensive audit logging of all data access
- Design data anonymization for analytics and reporting
- Implement secure authentication (JWT with refresh tokens, 2FA option)

Volatile Design Area: Authentication mechanisms, encryption algorithms, access control policies

Design Impact:

- Security layer in all system components
- Encrypted data at rest and in transit
- Authentication service with token management
- Audit log database design
- Regular security testing integration points

7. Payment Fraud and Transaction Security

Payment system vulnerabilities could lead to fraudulent transactions, chargebacks, or financial losses.

Impact: High - Financial loss, legal issues, user trust loss

Likelihood: Medium

Mitigation (Design) Strategies:

- Design webhook signature verification for all payment callbacks

- Implement idempotency for payment transactions
- Design fraud detection rules (unusual patterns, velocity checks)
- Create transaction reconciliation processes

Volatile Design Area: Payment gateway integration, fraud detection rules

Design Impact:

- Payment service abstraction with security focus
- Transaction state machine with validation
- Webhook verification service
- Payment reconciliation module

2.2.3 Requirement Change Risks

We found three risks that address evolving requirements for AI features, regulatory compliance, and user interface design.

8. AI Feature Requirements Evolution

Understanding of required AI capabilities may evolve during development based on user feedback and testing.

Impact: Medium - Development timeline affected, architectural changes needed

Likelihood: High

Mitigation (Design) Strategies:

- Design modular AI service architecture
- Create abstraction layer between AI engine and application logic
- Design configuration-based AI behavior (prompt templates, response formats)
- Implement feature flags for gradual AI capability rollout

Volatile Design Area: AI model architecture, NLP pipeline, triage rules, response generation

Design Impact:

- Plugin architecture for AI capabilities
- Configuration management system
- A/B testing framework
- Model versioning and rollback capabilities

9. Regulatory Compliance Changes

Healthcare regulations, data protection laws may change during or after development.

Impact: High - Legal compliance required, potential service interruption

Likelihood: Medium

Mitigation (Design) Strategies:

- Design configurable consent management system
- Create flexible data retention and deletion mechanisms
- Design audit trail for all patient data access
- Implement configurable disclaimers and terms of service

Volatile Design Area: Consent flows, data retention policies, audit requirements

Design Impact:

- Consent management module
- Data lifecycle management system
- Compliance configuration service
- Comprehensive audit logging

10. User Interface and Experience Changes

UX requirements may evolve based on user testing, accessibility needs, or cultural considerations.

Impact: Medium - Development effort, consistency challenges

Likelihood: High

Mitigation (Design) Strategies:

- Design component-based UI architecture (atomic design)
- Create design system with reusable components
- Implement theme system for easy visual changes
- Design responsive layouts adaptable to different screen sizes

Volatile Design Area: UI components, layouts, interaction patterns, color schemes

Design Impact:

- Component library design
- Theme configuration system
- Responsive grid system
- Accessibility layer

2.2.4 Integration Risks

External API changes pose integration risks requiring abstraction layers and monitoring.

11. Third-Party API Changes

External service providers may change their APIs, pricing models, or terms of service.

Impact: Medium-High - Integration breakage, cost increases, service migration needed

Likelihood: Medium

Mitigation (Design) Strategies:

- Design abstraction layer (adapter pattern) for all external APIs
- Create comprehensive integration test suites
- Design version-specific API clients with graceful upgrades
- Implement API monitoring and alerting

Volatile Design Area: API integration points, authentication methods, data formats

Design Impact:

- Service interface design independent of provider
- API version management system
- Integration testing framework
- Multiple provider support

3. System Architecture

3.1 System Level Architecture

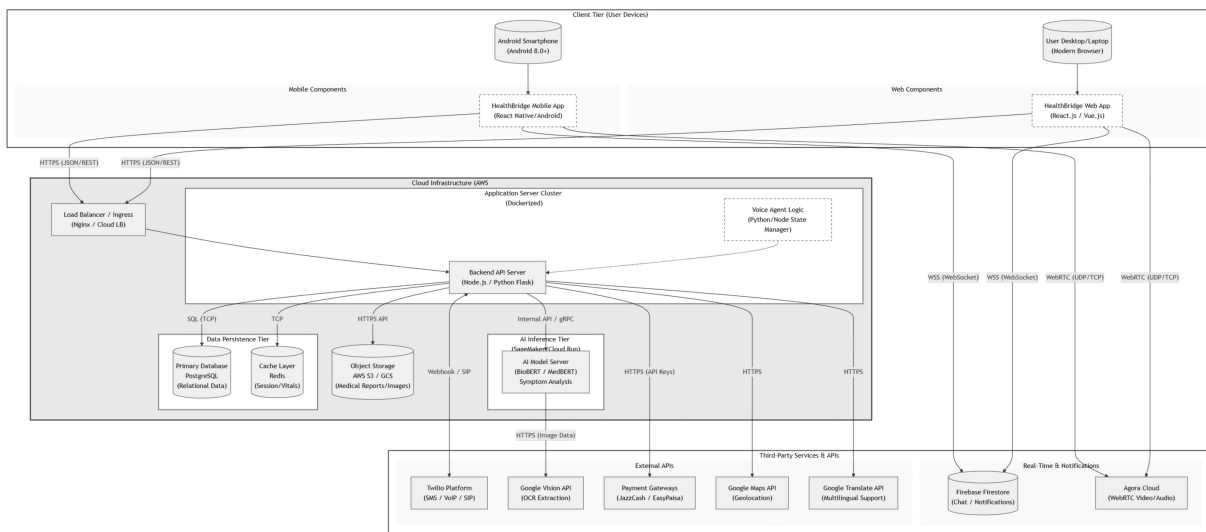


Figure 1 [Deployment Diagram](#)

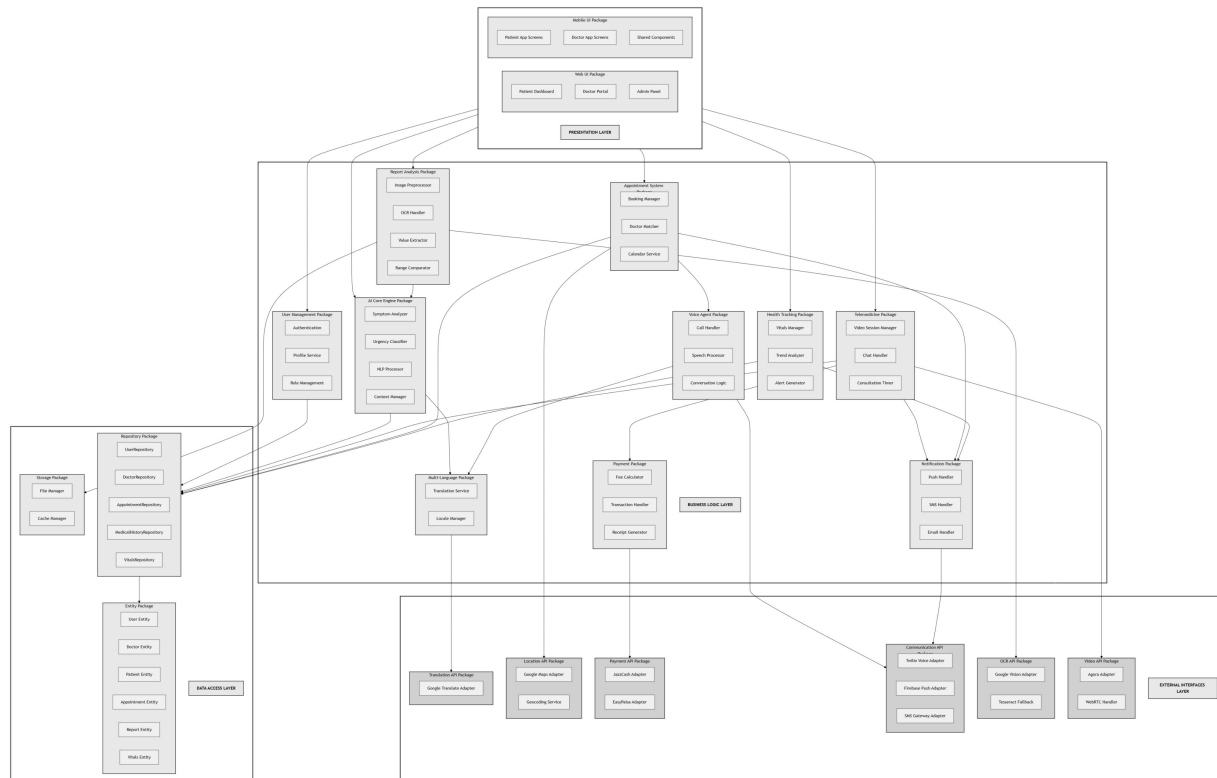


Figure 2 Package Diagram

3.2 Sub-System / Component / Module Level Architecture

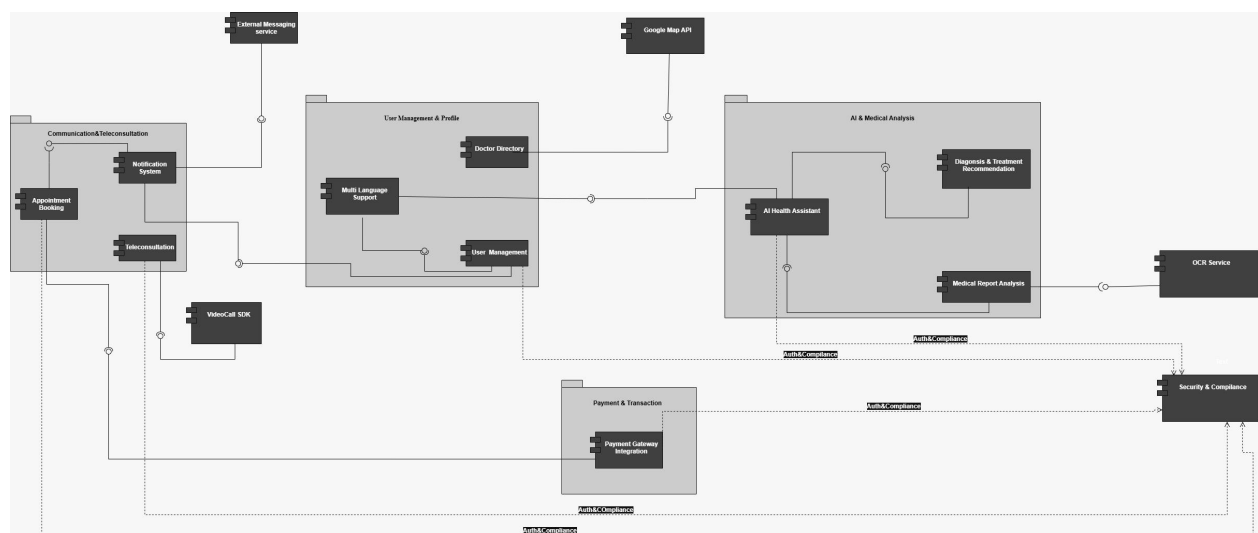


Figure 3 System Level Component Diagram

3.3 Sub-Component / Sub-Module Level Architecture (1...n)

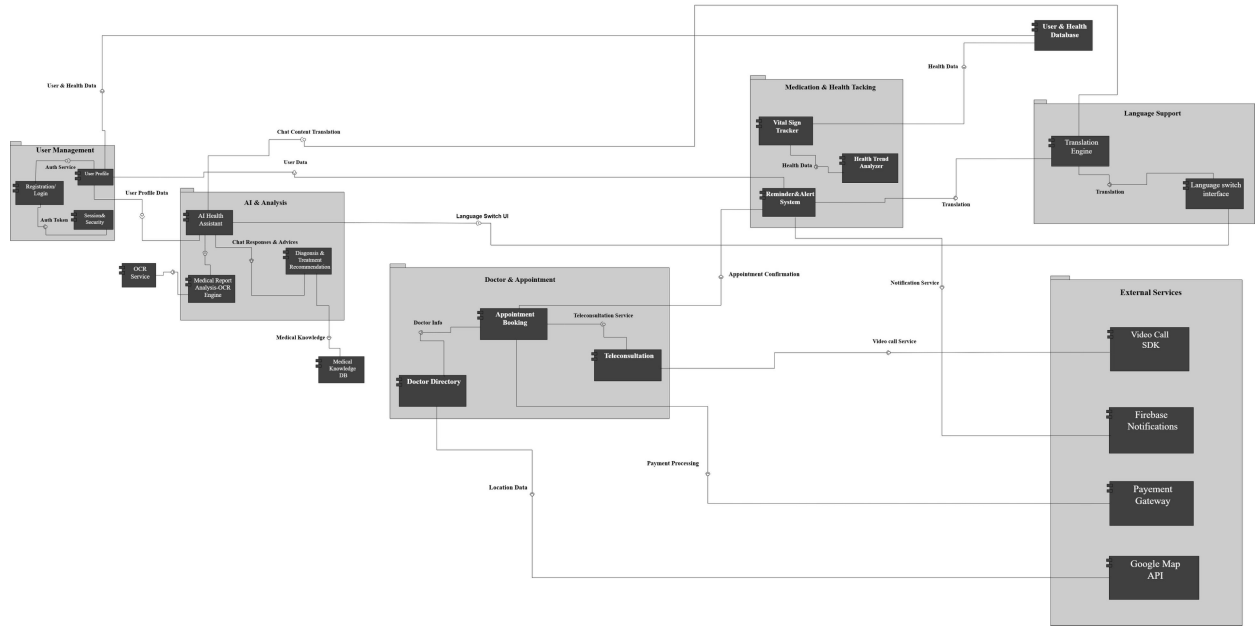


Figure 4 [Sub-System Level Component Diagram](#)

4. Design Strategies

This section outlines the key design strategies and architectural decisions that shape HealthBridge.ai's system organization. Each strategy addresses specific project requirements while considering future extensibility, maintainability, and scalability.

4.1 Layered Architecture Pattern

Strategy Description: HealthBridge.ai employs a three-tier layered architecture separating presentation, business logic, and data access layers.

Reasoning:

- Ensures clear separation of concerns, making each layer independently testable and maintainable
- Aligns with our objective to build a scalable healthcare platform where AI, consultation, and report analysis modules can evolve independently
- Facilitates team collaboration as frontend and backend developers can work in parallel

Trade-offs:

- *Advantage:* Changes in UI don't affect business logic; database modifications are isolated from application code

- *Disadvantage:* Additional complexity in managing inter-layer communication; potential performance overhead from layer transitions
- *Mitigation:* Use efficient data transfer objects (DTOs) and minimize unnecessary layer crossings

4.2 Microservices-Oriented Component Design

Strategy Description: Core functionalities are designed as independent, loosely-coupled service modules: AI Health Assistant Service, Medical Report Analysis Service, Appointment Management Service, Consultation Service, and Notification Service.

Reasoning:

- Each service addresses a distinct healthcare domain (AI diagnosis, report interpretation, doctor matching)
- Supports independent development, testing, and deployment of critical features
- Aligns with our Agile methodology where features are delivered incrementally across sprints
- Different services have different scaling needs (AI inference requires more compute than appointment booking)

Trade-offs:

- *Advantage:* Services can be scaled independently; failure in one service doesn't crash entire system; technology stack can vary per service (Python for AI, Node.js for real-time features)
- *Disadvantage:* Increased operational complexity; inter-service communication overhead; distributed transaction challenges
- *Mitigation:* Implement API gateway for unified access; use message queues for asynchronous operations; maintain service contracts

4.3 Domain-Driven Design for Healthcare Context

Strategy Description: System design reflects real-world healthcare domain concepts: Patient, Doctor, MedicalReport, Consultation, Diagnosis, Appointment as primary domain entities.

Reasoning:

- Healthcare workflows are complex and domain-specific (triage levels, specialist matching, medication schedules)
- Domain experts (doctors, healthcare workers) can easily understand and validate system behavior
- Business rules are encapsulated within domain objects (e.g., Appointment knows its validation rules)

Trade-offs:

- *Advantage:* Code directly maps to healthcare processes; easier communication with medical advisors; reduced misinterpretation of requirements
- *Disadvantage:* Learning curve for developers unfamiliar with healthcare domain; potential over-engineering for simple CRUD operations
- *Mitigation:* Comprehensive domain documentation; medical terminology glossary; regular stakeholder reviews

System Reuse: Domain models can be reused across mobile app, web platform, and future admin dashboard.

4.4 Repository Pattern for Data Management

Strategy Description: Data access is abstracted through repository interfaces (UserRepository, MedicalReportRepository, AppointmentRepository) that encapsulate database operations.

Reasoning:

- Decouples business logic from database implementation details
- Supports our requirement for PostgreSQL as primary database while allowing future migration flexibility
- Simplifies unit testing through repository mocking
- Centralizes data access logic, ensuring consistent query patterns

Trade-offs:

- *Advantage:* Database can be switched (PostgreSQL to MongoDB) with minimal business logic changes; complex queries are reusable; easy to add caching layer
- *Disadvantage:* Additional abstraction layer; potential performance impact for simple queries
- *Mitigation:* Use repository only for complex entities; allow direct queries for simple read operations

Data Persistence Strategy:

- **PostgreSQL:** Structured data (users, appointments, consultations, medications)
- **Redis:** Session management, caching frequently accessed data (doctor profiles, medical terminology)
- **Cloud Storage (Google Cloud Storage):** Large binary files (medical report PDFs, images)
- **Vector Database:** AI model embeddings for semantic search in medical knowledge base

4.5 API Gateway Pattern for Centralized Access

Strategy Description: All client requests route through a single API Gateway that handles authentication, rate limiting, request routing, and response aggregation.

Reasoning:

- Provides single entry point for web and mobile clients
- Centralizes cross-cutting concerns (authentication, logging, monitoring)
- Shields internal microservices architecture from clients
- Supports our security requirements for JWT authentication and HTTPS communication

Trade-offs:

- *Advantage:* Simplified client-side code; centralized security enforcement; easier API versioning; unified logging
- *Disadvantage:* API Gateway becomes single point of failure; potential bottleneck under high load

- *Mitigation:* Deploy multiple gateway instances with load balancer; implement circuit breakers; cache responses where appropriate

Future Extension: New API versions (v2) can coexist with v1 through gateway routing rules.

4.6 Strategy Pattern for AI Model Management

Strategy Description: AI Health Assistant uses Strategy pattern to support multiple NLP models and diagnosis algorithms interchangeably.

Reasoning:

- Core AI functionality must evolve as better models become available
- Different models may be optimal for different tasks (symptom analysis vs. medication suggestions)
- Supports A/B testing of model improvements before full deployment
- Aligns with our requirement for continuous AI model refinement based on user feedback

Trade-offs:

- *Advantage:* Easy model switching without code changes; parallel evaluation of multiple models; gradual rollout of new models; fallback to simpler models if advanced models fail
- *Disadvantage:* Increased complexity in model versioning and configuration management
- *Mitigation:* Centralized model registry; version-controlled model configurations; automated model performance monitoring

Implementation:

AIModelStrategy Interface

- SymptomAnalysisModel (primary diagnosis engine)
- TriageClassificationModel (urgency level detection)
- MedicalTermExplanationModel (simplification engine)

4.7 Adapter Pattern for Third-Party Integrations

Strategy Description: External services (payment gateways, video SDKs, OCR services, SMS providers) are accessed through adapter interfaces that abstract provider-specific implementations.

Reasoning:

- Healthcare platform depends on multiple external services with potential reliability issues
- Service providers may change APIs, pricing, or availability
- Supports our requirement for multiple payment methods (JazzCash, EasyPaisa, bank transfers)
- Allows switching providers without affecting business logic

Trade-offs:

- *Advantage:* Easy provider switching; support multiple concurrent providers; graceful fallback when primary service fails
- *Disadvantage:* Additional abstraction complexity; lowest-common-denominator interface may not expose all provider features
- *Mitigation:* Design adapters for 80% use cases; allow provider-specific extensions when needed

Key Adapters:

- **PaymentGatewayAdapter:** JazzCashAdapter, EasyPaisaAdapter, BankTransferAdapter
- **VideoCallAdapter:** AgoraAdapter, TwilioAdapter
- **OCRAdapter:** GoogleVisionAdapter
- **NotificationAdapter:** SMSAdapter, EmailAdapter, PushNotificationAdapter

4.8 Observer Pattern for Real-Time Updates

Strategy Description: System uses Observer pattern with WebSocket connections for real-time notifications (appointment updates, chat messages, consultation requests, medication reminders).

Reasoning:

- Healthcare scenarios require immediate notifications (doctor accepts appointment, emergency triage result)
- Reduces server polling overhead and improves user experience
- Supports our requirement for real-time chat during consultations
- Enables instant medication reminder delivery

Trade-offs:

- *Advantage:* Instant updates without page refresh; reduced server load compared to polling; better user experience
- *Disadvantage:* Persistent connections consume server resources; connection management complexity; handling disconnections and reconnections
- *Mitigation:* Connection pooling; automatic reconnection logic; fallback to periodic polling for less critical updates

Implementation:

- WebSocket server maintains active connections
- Event publisher notifies subscribers when domain events occur (AppointmentConfirmed, MessageReceived, ReportAnalyzed)
- Clients subscribe to relevant event channels based on user role and context

4.9 Caching Strategy for Performance Optimization

Strategy Description: Multi-level caching strategy: CDN for static assets, Redis for application data, database query result caching.

Reasoning:

- AI model inference is computationally expensive; caching common symptom-diagnosis patterns improves response time
- Doctor profiles and medical terminology are frequently accessed but rarely change
- Medical reports take time to analyze; caching results for same report prevents redundant processing
- Supports our NFR requirement for AI response within few seconds

Trade-offs:

- *Advantage:* Dramatically reduced response times; lower database load; reduced AI API costs for duplicate queries
- *Disadvantage:* Cache invalidation complexity; stale data risks; increased memory usage
- *Mitigation:* Implement time-based expiration (TTL); use cache-aside pattern; invalidate cache on data updates

Caching Layers:

1. **Redis Application Cache:** User sessions (30 min TTL), doctor profiles (1 hour TTL), common AI responses (6 hours TTL)
2. **Database Query Cache:** Frequently accessed reports (24 hours TTL)
3. **Client-Side Cache:** Medical terminology glossary, UI components (indefinite with version invalidation)

4.10 Asynchronous Processing for Long-Running Operations

Strategy Description: Long-running tasks (OCR processing, AI model inference, email notifications, report generation) are processed asynchronously using message queues (RabbitMQ/Redis Queue).

Reasoning:

- Medical report OCR can take 10-30 seconds for large PDFs
- AI diagnosis with multiple models requires sequential processing
- Video call recordings and consultation summaries generation are time-intensive
- Supports our NFR requirement for system responsiveness under load

Trade-offs:

- *Advantage:* Immediate user feedback; system remains responsive during heavy processing; failed jobs can be retried automatically; background tasks don't block user interactions
- *Disadvantage:* Eventual consistency challenges; complex error handling; job monitoring overhead

- *Mitigation:* Job status tracking in database; user notifications on completion; retry with exponential backoff; dead letter queues for failed jobs

Queue Design:

- **High Priority Queue:** Urgent triage results, emergency notifications (processed immediately)
- **Medium Priority Queue:** Report analysis, AI responses (processed within 5 seconds)
- **Low Priority Queue:** Email sending, analytics updates, report generation (processed within minutes)

4.11 Security-First Design Approach

Strategy Description: Security is embedded at every architectural layer: encrypted data storage, HTTPS-only communication, JWT authentication, role-based access control, audit logging.

Reasoning:

- Healthcare data is highly sensitive and regulated
- Patient trust depends on data confidentiality and integrity
- Supports our NFR requirements for encrypted passwords, secure communication, input validation
- Prepares system for future compliance certifications

Trade-offs:

- *Advantage:* Comprehensive protection against common attacks; audit trail for compliance; user trust and confidence
- *Disadvantage:* Performance overhead from encryption/decryption; increased development complexity; strict validation may affect user experience
- *Mitigation:* Use hardware acceleration for encryption; balance security with usability; clear error messages for security failures

Security Mechanisms:

- **Authentication:** JWT tokens with refresh mechanism, secure password hashing (bcrypt), optional 2FA
- **Authorization:** Role-based access control (Patient, Doctor, Admin), permission checks at API gateway and service levels
- **Data Protection:** AES-256 encryption for sensitive fields (medical history, payment info), TLS 1.3 for data in transit
- **Input Validation:** Sanitization of all user inputs, protection against SQL injection, XSS, CSRF
- **Audit Logging:** Comprehensive logs of data access, modifications, authentication attempts

4.12 Responsive and Progressive User Interface Paradigm

Strategy Description: Component-based UI architecture using React.js with responsive design, progressive web app (PWA) capabilities, and mobile-first approach.

Reasoning:

- Users access system from diverse devices (smartphones, tablets, desktops)
- Pakistan has varying internet speeds; offline capability improves accessibility
- Supports our user characteristics requirement for low-literacy users with voice support
- Dual-language support (English, Urdu) requires flexible UI components

Trade-offs:

- *Advantage:* Single codebase for multiple devices; offline functionality for critical features; installable as app without app store; fast initial load with lazy loading
- *Disadvantage:* Complex state management; larger initial bundle size; PWA limitations compared to native apps
- *Mitigation:* Code splitting for faster load times; service workers for offline support; clear offline indicators

UI Design Principles:

- **Atomic Design:** Reusable components (Button, Input, Card) composed into larger interfaces
- **Responsive Grid:** Mobile-first breakpoints ensuring usability on small screens
- **Progressive Enhancement:** Core functionality works without JavaScript; enhanced features layered on top
- **Accessibility:** ARIA labels, keyboard navigation, screen reader support, high contrast mode
- **Internationalization:** Language switching without page reload; RTL support for Urdu; culturally appropriate icons

4.13 Concurrency and Data Consistency Strategy

Strategy Description: Optimistic locking for appointment booking, pessimistic locking for payment transactions, eventual consistency for non-critical data, event sourcing for audit trail.

Reasoning:

- Multiple users may attempt to book same doctor appointment slot simultaneously
- Payment transactions require strict consistency (no double charges)
- Medical record updates must maintain data integrity
- Consultation history needs complete audit trail for legal/medical purposes

Trade-offs:

- *Advantage:* High concurrency for read operations; data integrity for critical transactions; complete history for troubleshooting
- *Disadvantage:* Complex conflict resolution; potential user frustration from booking conflicts; increased storage for event logs
- *Mitigation:* Clear user feedback on conflicts; automatic retry with alternative suggestions; log rotation policies

Concurrency Handling:

- **Appointment Booking:** Optimistic locking with version numbers; conflict resolution shows alternative time slots
- **Payment Processing:** Pessimistic database locks; idempotent transaction IDs; two-phase commit for distributed transactions
- **Medical Records:** Last-write-wins with conflict detection; manual merge interface for critical conflicts
- **Chat Messages:** Append-only event log; eventual consistency acceptable for message delivery

4.14 Modular Monolith with Microservices Evolution Path

Strategy Description: Initial deployment as modular monolith with clear module boundaries, designed for future extraction into microservices as system scales.

Reasoning:

- 8-month academic timeline favors simpler deployment initially
- Four-person team can manage monolithic development more easily
- Module boundaries prepare for future microservices split
- Supports our Agile methodology with incremental feature delivery

Trade-offs:

- *Advantage:* Simpler deployment and debugging; faster initial development; lower infrastructure costs; easier transaction management
- *Disadvantage:* Entire system must be deployed for any change; scaling requires scaling entire application; technology stack is uniform
- *Mitigation:* Strong module boundaries using packages; dependency injection for loose coupling; separate databases per module to enable future extraction

Module Organization:

Monolithic Application

- AuthModule (independent, extractable)
- AIHealthAssistantModule (compute-intensive, future microservice)
- ReportAnalysisModule (compute-intensive, future microservice)
- ConsultationModule (real-time requirements, future microservice)
- AppointmentModule (high read volume, future microservice)
- NotificationModule (asynchronous, future microservice)
- PaymentModule (PCI compliance, future microservice)

Evolution Strategy: Modules designed with well-defined APIs; when scale demands, extract modules to separate services without changing business logic.

Summary of Strategic Trade-offs

These design strategies collectively address HealthBridge.ai's core requirements:

1. **Scalability:** Microservices orientation, caching, asynchronous processing, modular design
2. **Maintainability:** Layered architecture, repository pattern, clear module boundaries
3. **Extensibility:** Strategy pattern for AI, adapter pattern for integrations, plugin architecture
4. **Security:** Security-first approach, encryption, authentication, audit logging
5. **Performance:** Multi-level caching, asynchronous processing, optimized data access
6. **Usability:** Responsive UI, progressive enhancement, real-time updates, dual-language support
7. **Reliability:** Graceful degradation, fallback mechanisms, retry logic, circuit breakers

Each strategy involves trade-offs between complexity and flexibility, performance and maintainability, development speed and long-term scalability. The chosen strategies balance these concerns while aligning with our 8-month timeline, four-person team capacity, and academic project constraints.

4. Detailed System Design

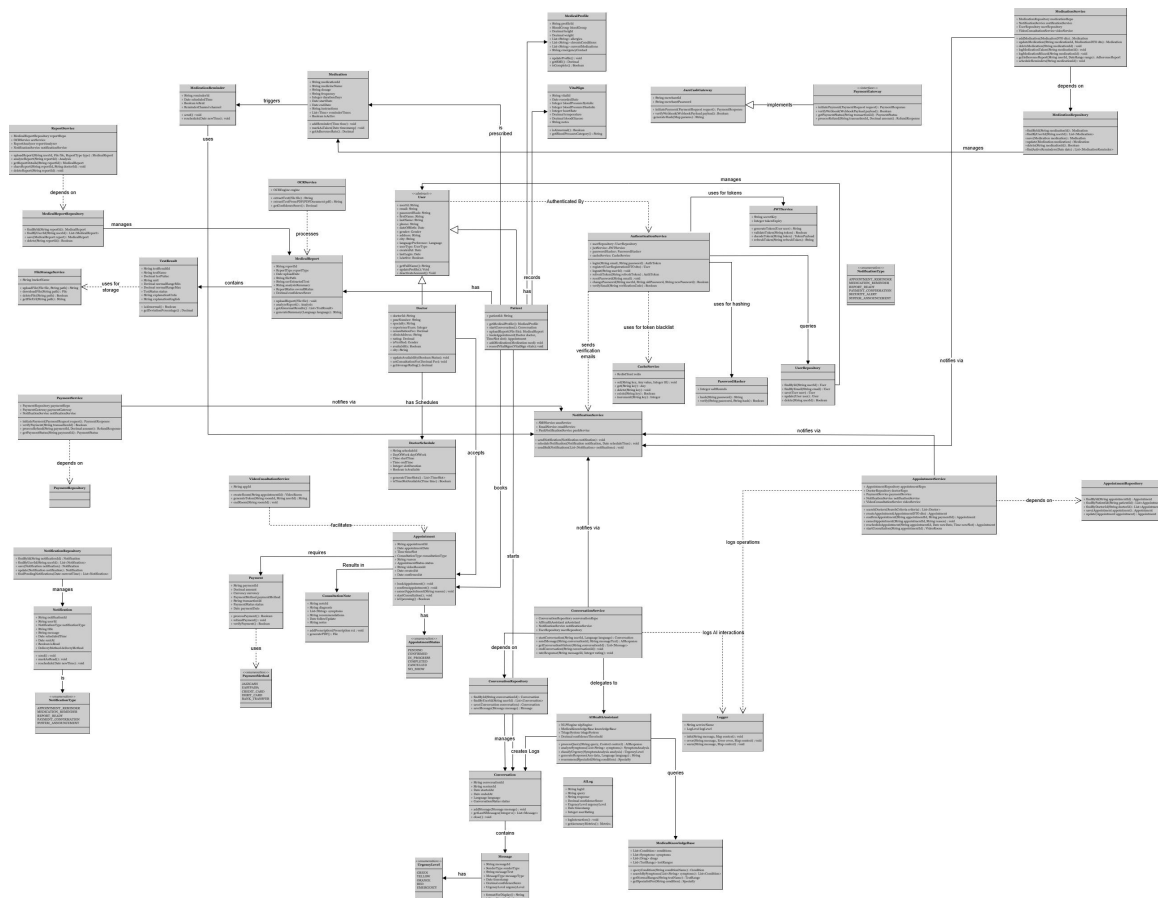


Figure 5 **Class Diagram**

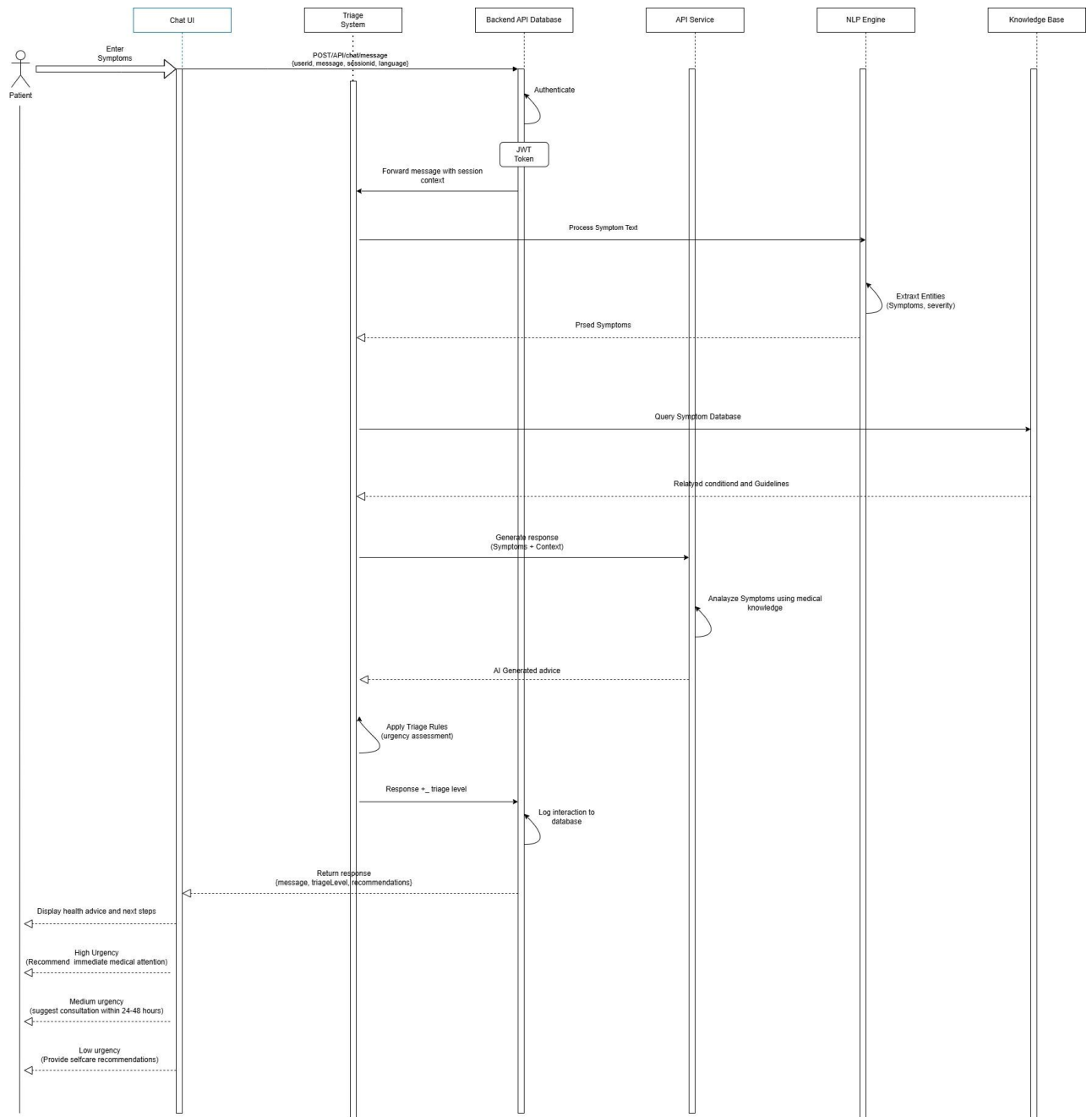


Figure 6 [Patient uses AI Health Assistant for symptom Check](#)

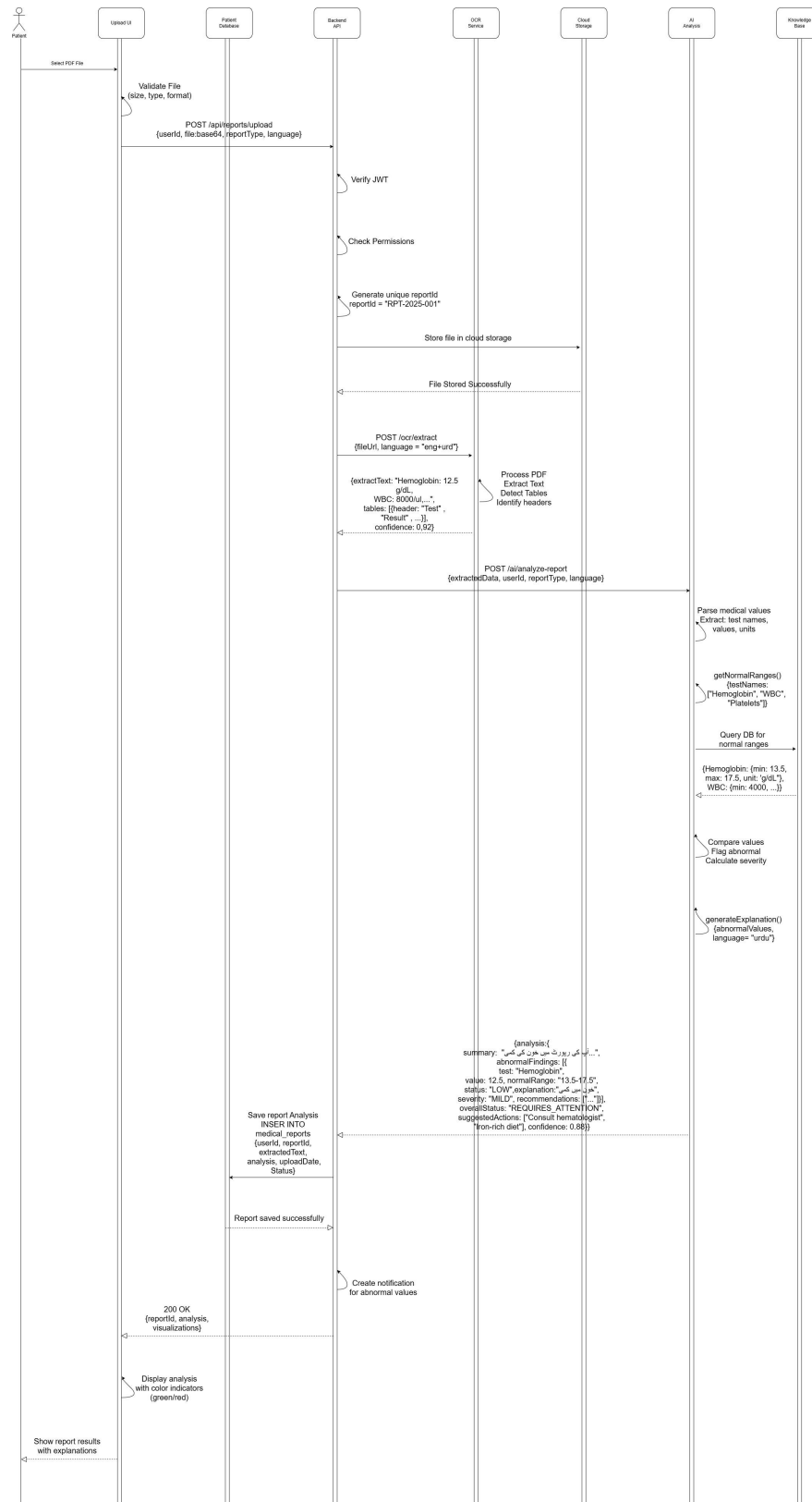


Figure 7 Patient uploads Medical Report for Analysis

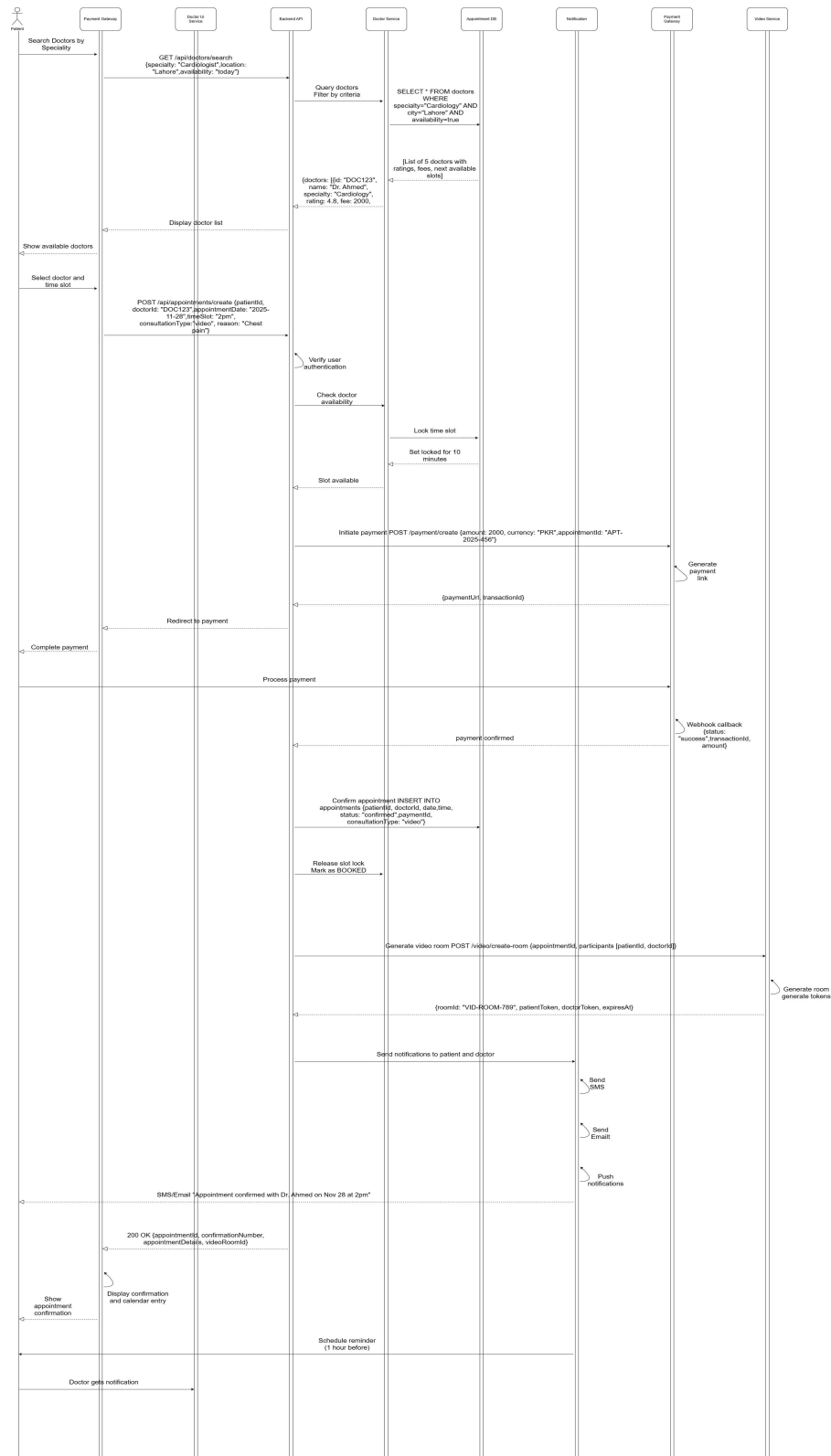


Figure 8 Patient books video consultation with doctor

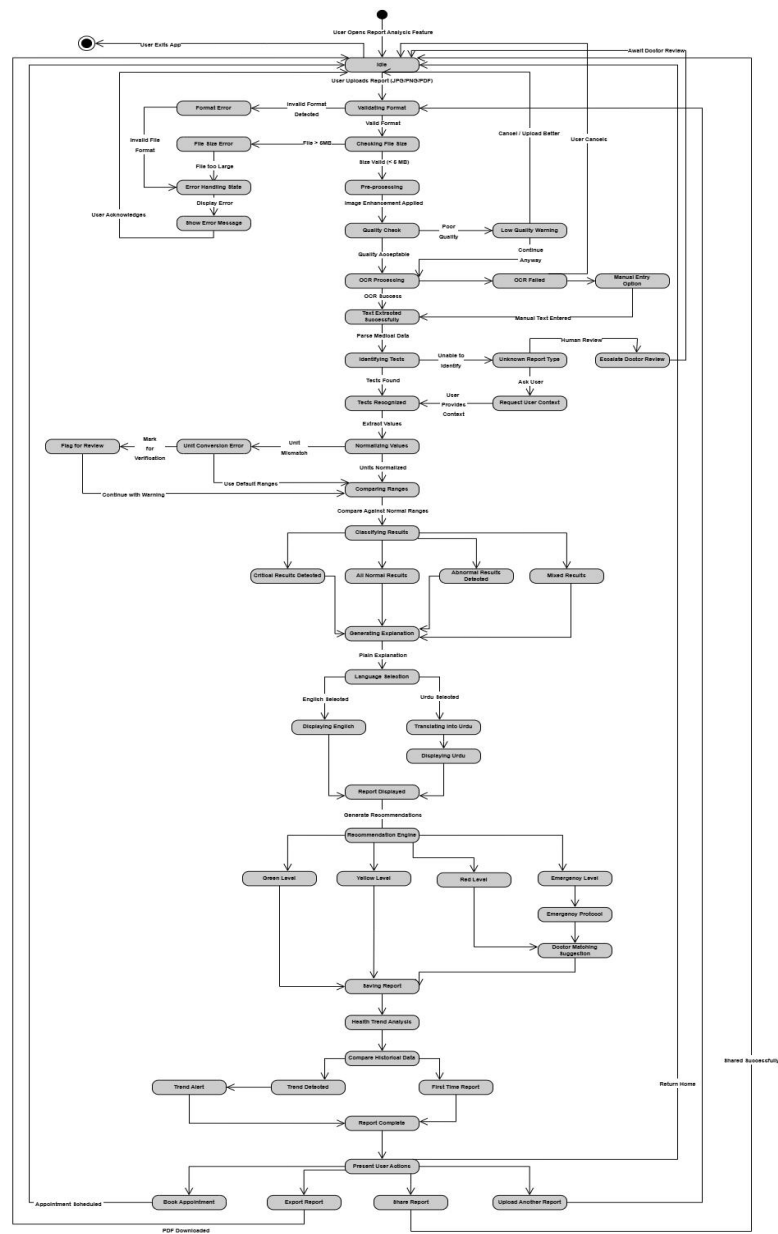


Figure 9 [Medical Report Analysis State Transition Diagram](#)

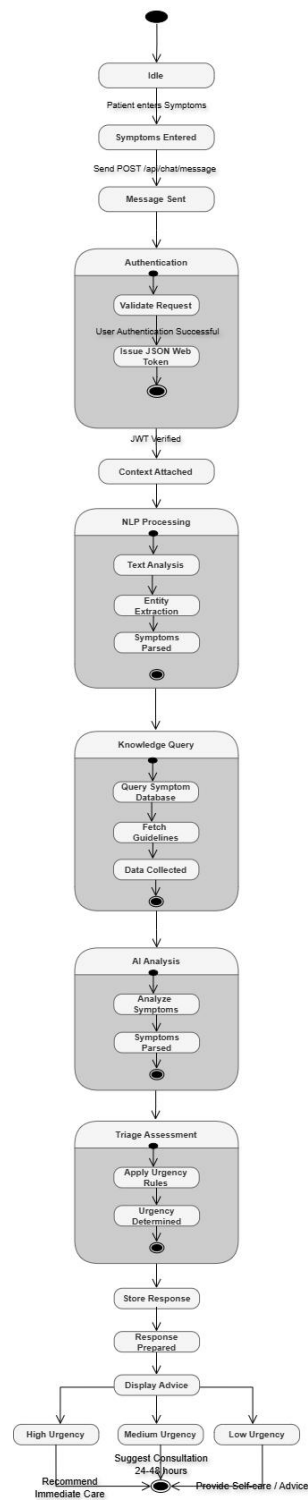


Figure 10 [Health Assistant Chat with Patient](#)

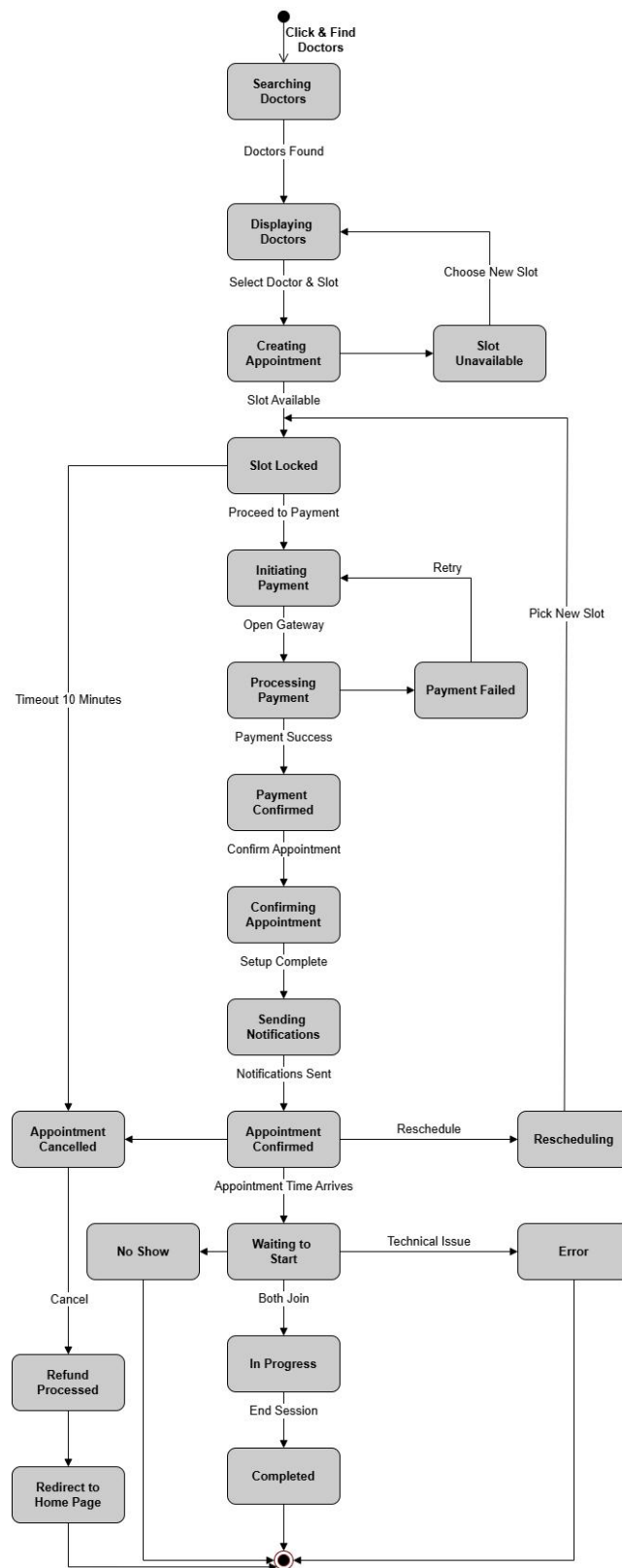


Figure 11 [Appointment Lifecycle State Transition Diagram](#)

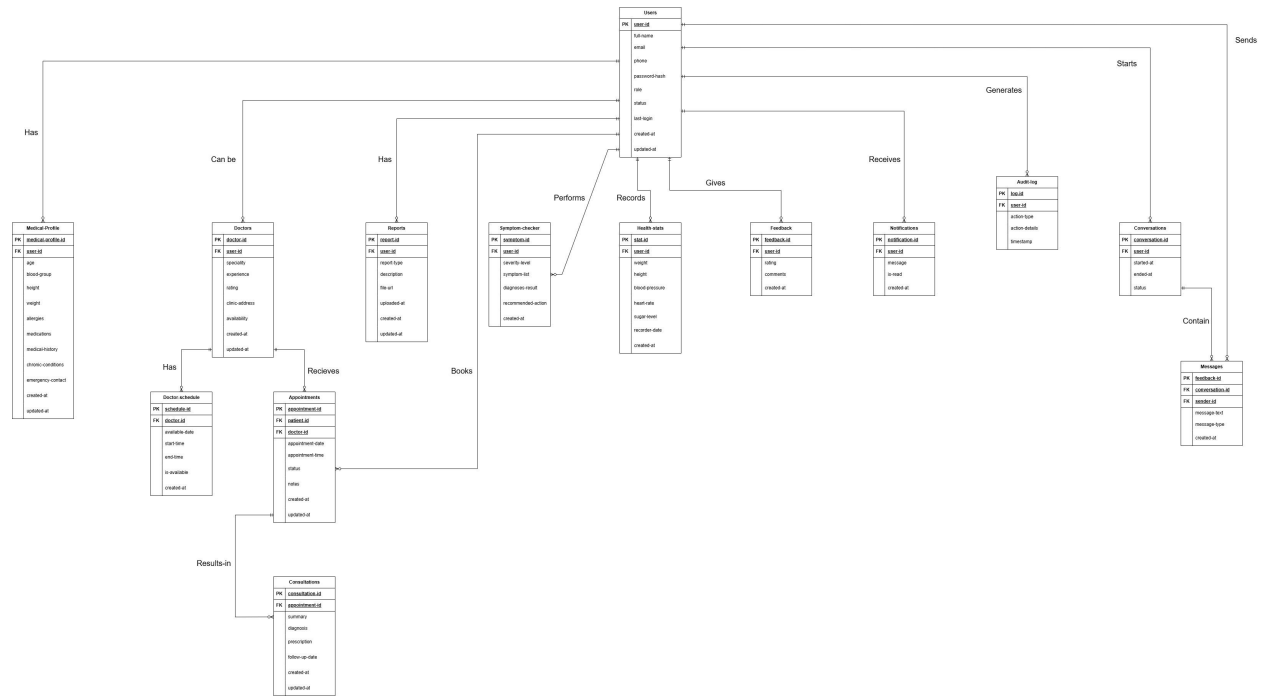


Figure 12 [Entity Relationship Diagram \(Logical Data Model\)](#)

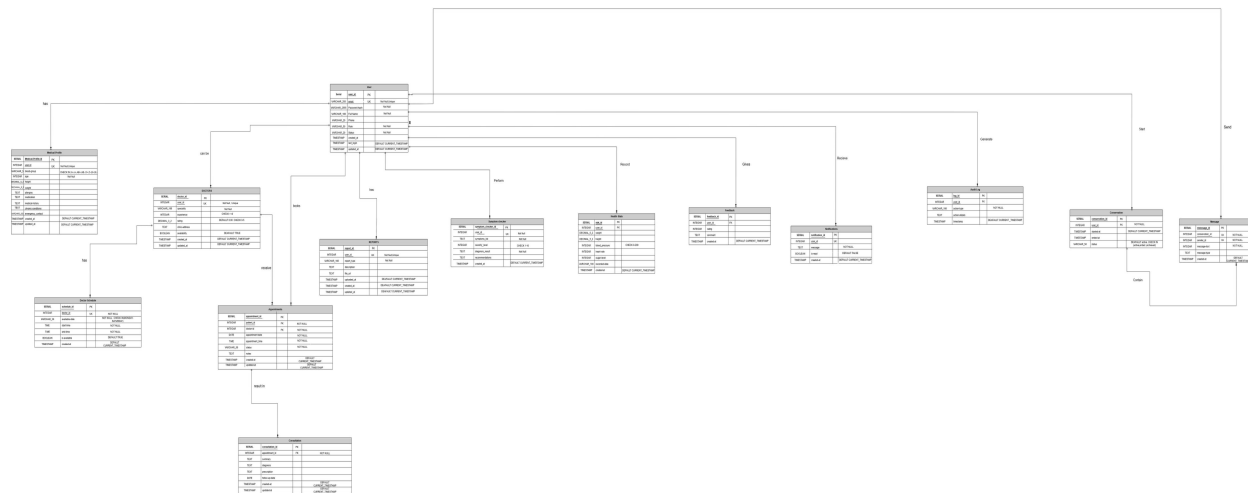


Figure 13 [Physical Data Model](#)

Detailed GUIs

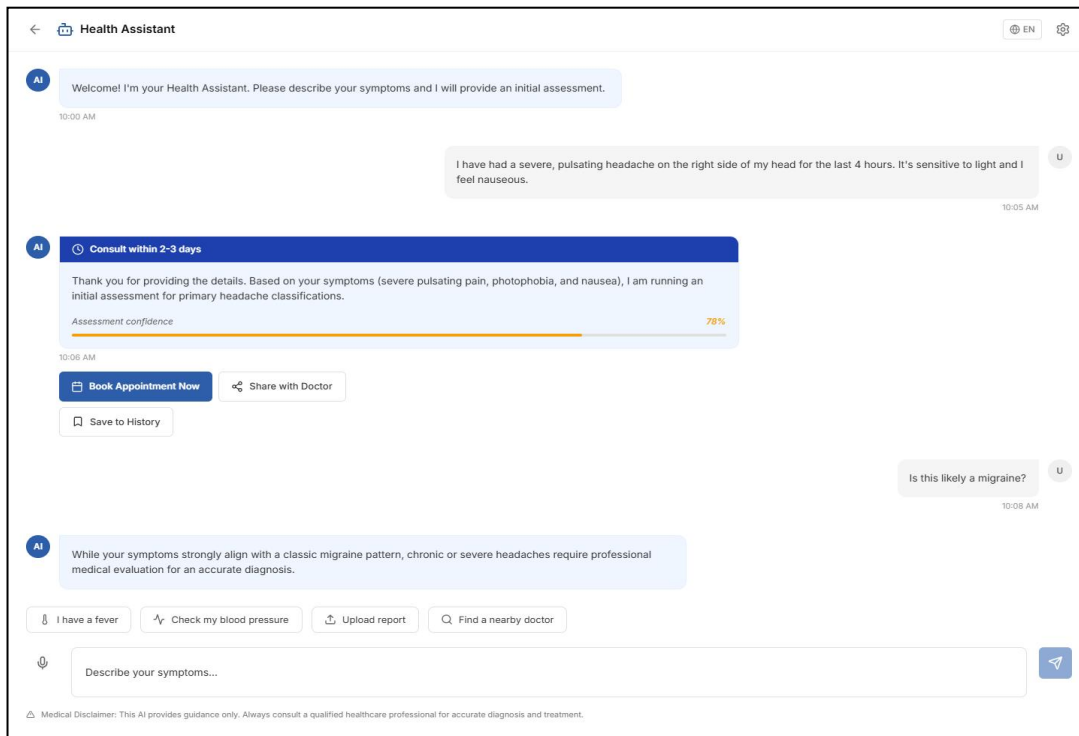


Figure 14: Health Assistant Chat

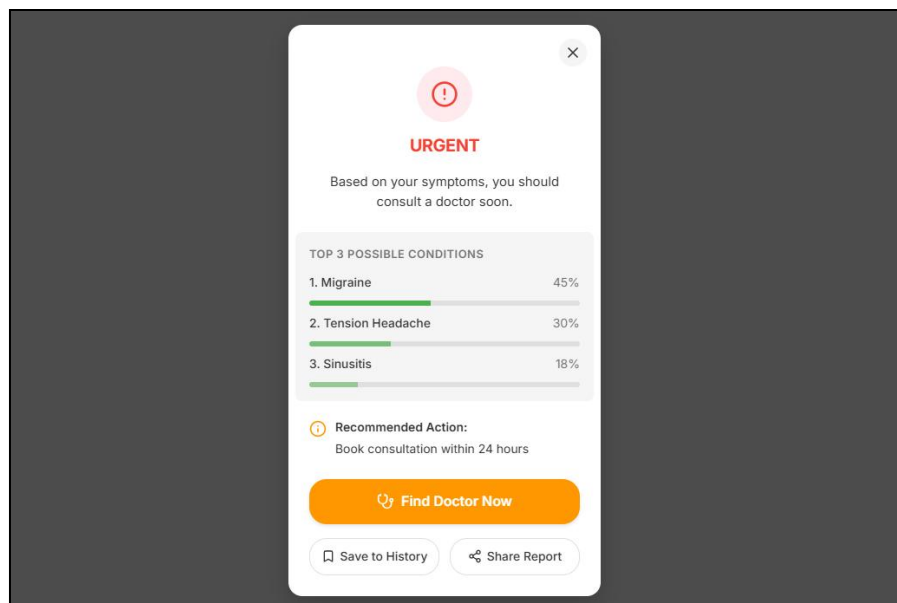


Figure 15: Urgency Result

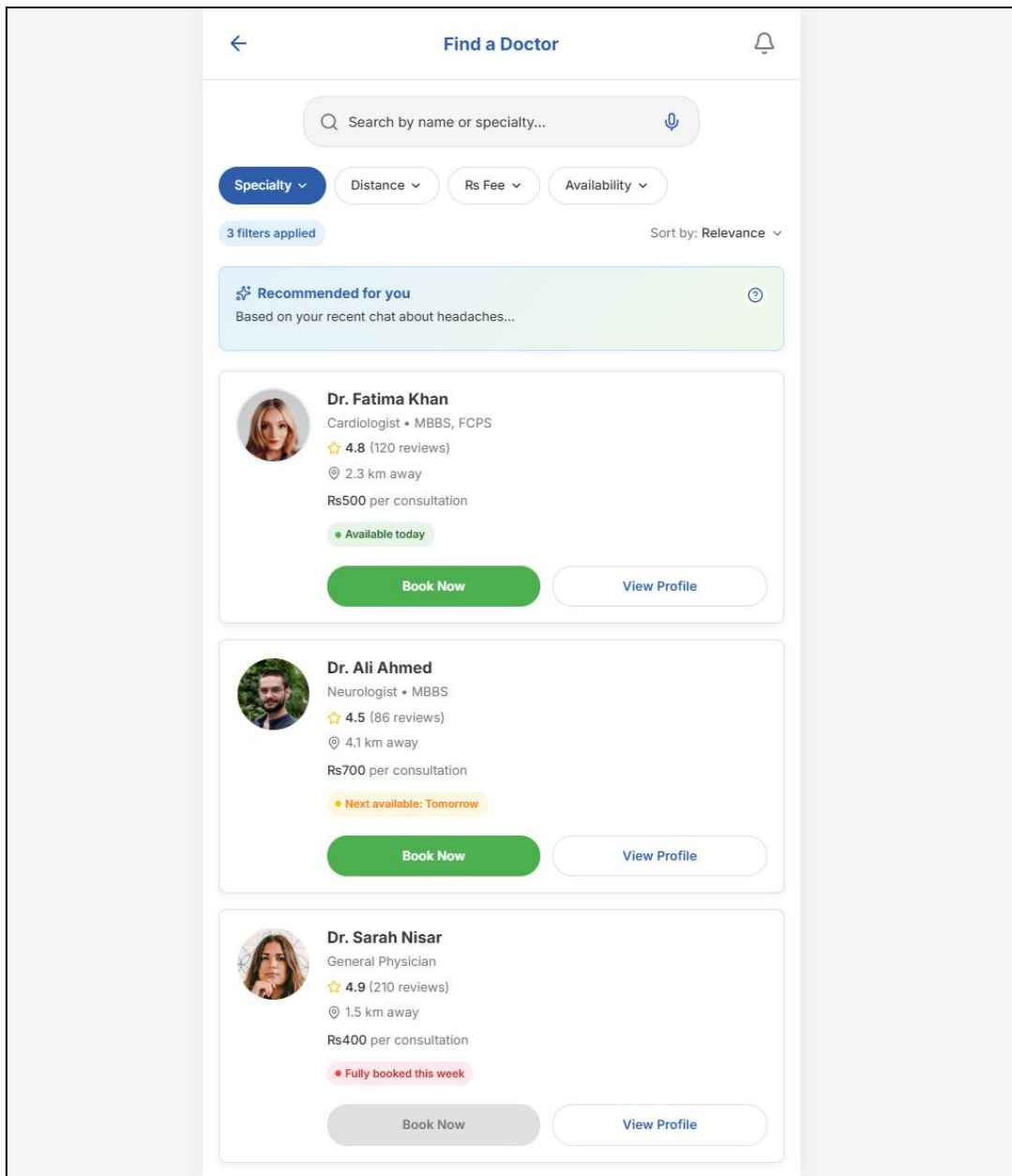
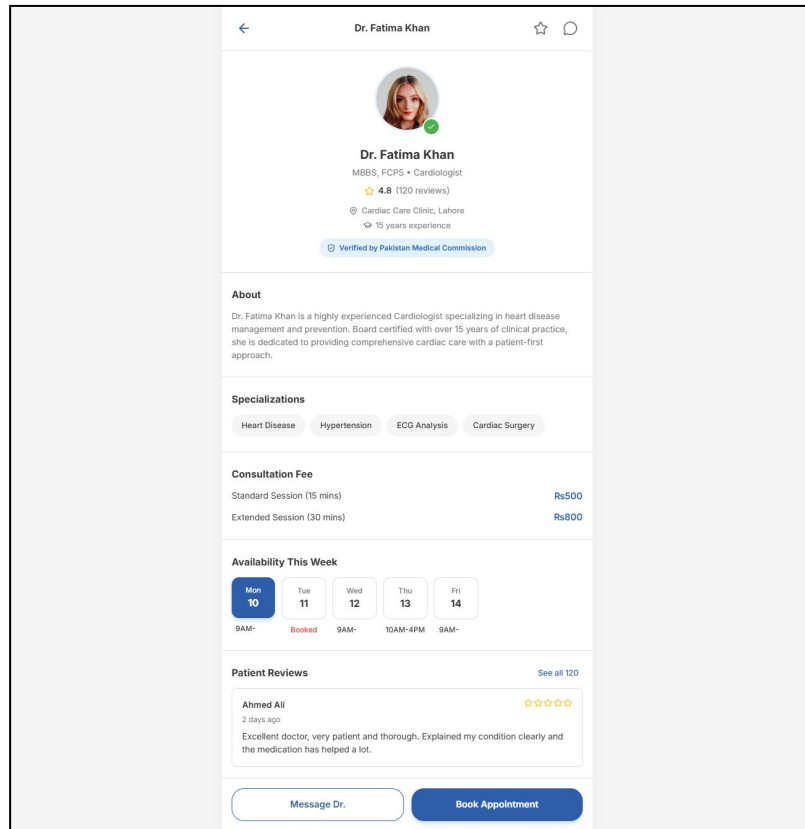
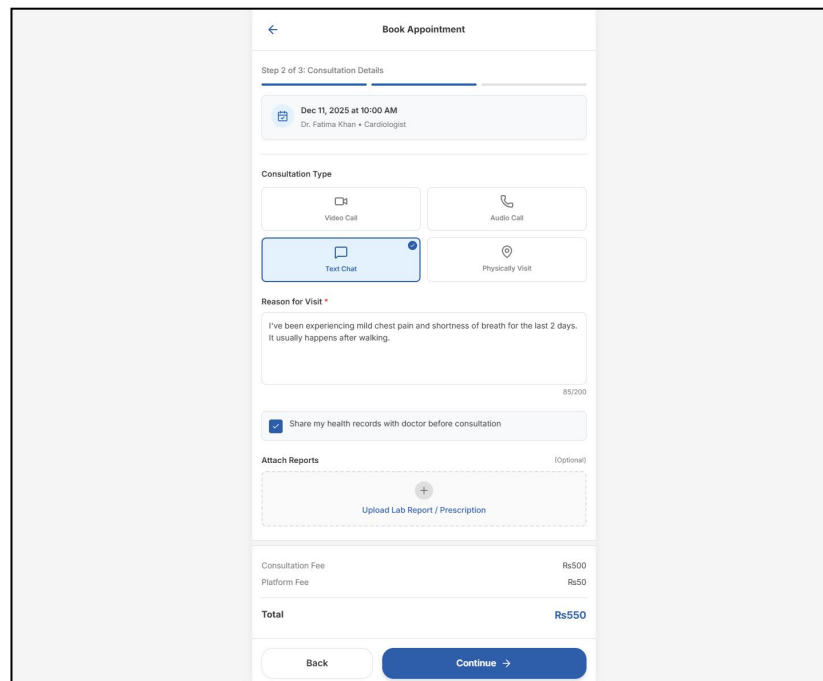


Figure 16: Doctor Search

**Figure 17: Doctor Profile****Figure 18: Booking Details**

The screenshot shows a mobile app interface for booking an appointment with Dr. Fatima Khan. The title bar at the top says "Book with Dr. Fatima Khan" with a back arrow on the left. Below the title bar, it says "Step 1 of 3: Choose Date & Time". The main content area shows a calendar for December 2025. The days of the week are listed at the top: Mon, Tue, Wed, Thu, Fri. The dates 09, 10, 11, 12, and 13 are shown below. The date 10 is selected and highlighted in blue. The date 12 is marked as "Booked" in red. Below the calendar, there are sections for "Available Time Slots (Dec 10)". The slots are categorized into MORNING, AFTERNOON, and EVENING. The MORNING slots are 09:00 AM, 09:30 AM, 10:00 AM (selected), 10:30 AM, 11:00 AM, and 11:30 AM. The AFTERNOON slots are 02:00 PM, 02:30 PM, and 03:00 PM. The EVENING slots are 05:00 PM, 06:00 PM, and a "Booked" slot. At the bottom, there is a blue button labeled "Continue →".

Figure 19: Booking Date and Time

The screenshot shows a mobile app interface for confirming and paying for an appointment. The title bar at the top says "Confirm & Pay" with a back arrow on the left. Below the title bar, it says "Step 3 of 3: Payment". The main content area shows an "Appointment Summary" section with a profile picture of Dr. Fatima Khan, a Cardiologist. The summary includes the date and time (Dec 11, 2025, 2:00 PM), the consultation type (Text Chat), and the reason for visit (Chest pain and shortness of breath...). Below the summary, there is a "Payment Method" section with four options: JazzCash, EasyPaisha (selected), Credit / Debit Card, and Cash Payment (Available for physical visits only). At the bottom, there is a checkbox for "I agree to the Terms & Conditions and Privacy Policy." and a blue button labeled "Confirm & Pay Rs550".

Figure 20: Payment Confirmation

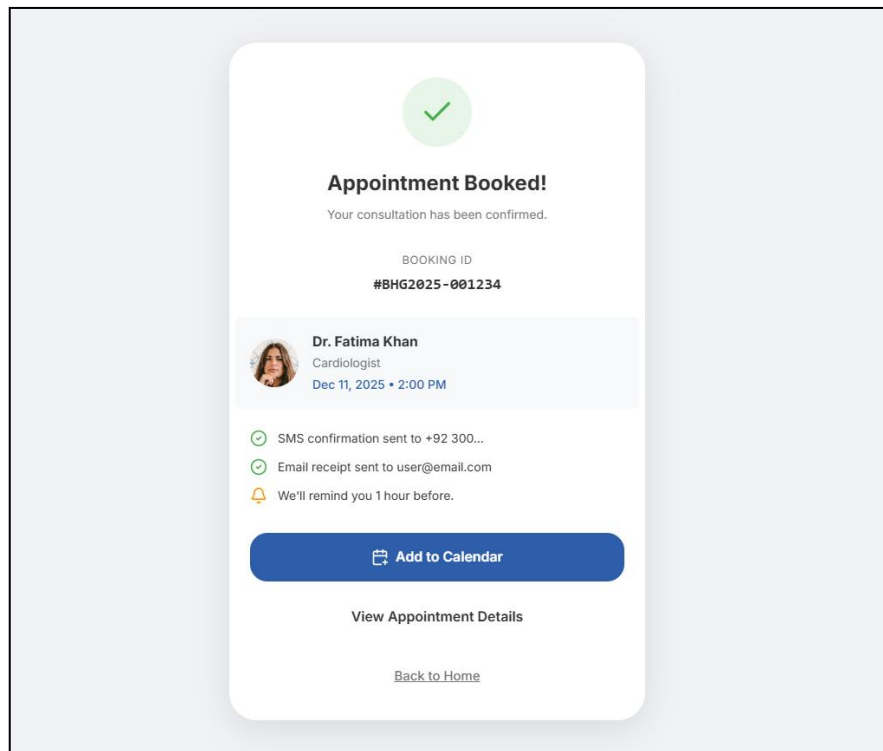


Figure 21: Successful Confirmation

5. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
PGBH01-2025-Proposal	Project Proposal	Oct 20, 2025	https://github.com/AqsaShahzad04/CP-1.git
PGBH01-2025-FS	Functional Specification	Oct 20, 2025	https://github.com/AqsaShahzad04/CP-1/blob/master/Approved%20SRS%20.pdf
PGBH01-2025-[1]	IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)	1998	https://ieeexplore.ieee.org/document/720574
PGBH01-2025-[2]	Software Engineering: A Practitioner's Approach (7th Edition) by Roger S. Pressman	2010	McGraw-Hill, New York
PGBH01-2025-[3]	Maternal Mortality in Pakistan: Challenges, Efforts, and Solutions	2022	https://pmc.ncbi.nlm.nih.gov/articles/PMC9420499/
PGBH01-2025-[4]	Oladoc – Book Doctors & Healthcare Professionals Online	2025	https://oladoc.com/
PGBH01-2025-[5]	Marham: Healthcare Services in Pakistan	2025	https://marham.pk/
PGBH01-2025-[6]	Sehat Kahani – Telehealth in Pakistan	2025	https://sehatkahani.com/
PGBH01-2025-[7]	Socioeconomic and Geographical Disparities in Healthcare Quality in Pakistan	2025	https://jpma.org.pk/index.php/public_html/article/view/21573
PGBH01-2025-[8]	Access to Primary Health Care in Pakistan	2022	https://mmi.edu.pk/blog/access-to-primary-health-care-in-pakistan/
PGBH01-2025-[9]	Original Diagram Files	2025	CP-1/originalDesigns at master · AqsaShahzad04/CP-1 · GitHub